

Betsy -- Autonomous Procurement Agent

Betsy is an AI agent that monitors inventory, detects procurement problems, and either acts on them autonomously or escalates them to a human for approval. Built as a school research project exploring what it takes to make an AI system that a non-technical operations manager can actually trust.

What it does

Betsy runs a procurement pipeline every 30 minutes (or on demand) and handles four failure modes:

- Stockout risk -- stock below reorder point, agent generates a PO to the best-scoring supplier
- Price spike -- unit price significantly above historical average, flags for human review
- Duplicate invoice -- same supplier, same amount, same period -- flags as potential fraud
- Supplier unavailable -- finds the next best alternative automatically

Any decision above the \$5,000 autonomous spending limit is held in an approval queue for a human to approve or decline. Everything below it executes automatically and logs to the audit trail.

Stack

Layer		Tech

Agent framework		LangGraph (Pipeline + Orchestra patterns)

LLM		Ollama -- llama3.1:8b (local)

Backend		FastAPI + uvicorn

Persistence		SQLite (built-in sqlite3)

Scheduling		APScheduler BackgroundScheduler

Frontend		Vanilla JS, no framework

Architecture at a glance

Who uses Betsy, and the parts inside it (a C4-style container view):

```
flowchart TB
    jenny["Jenny<br/><i>Operations manager</i>"]
    finance["Finance<br/><i>person</i>"]
    suppliers["Suppliers<br/><i>external</i>"]
    ollama["Ollama -- local AI<br/><i>llama3.1:8b</i>"]

    subgraph BETSY["Betsy &nbsp;[ software system ]"]
        direction TB
        betsyUI["Betsy dashboard<br/><i>betsy.html</i>"]
    end
```

```

devUI["Dev dashboard<br/><i>index.html</i>"]
api["Server<br/><i>FastAPI</i>"]
db["Database<br/><i>SQLite</i>"]
scheduler["Scheduler<br/><i>APScheduler</i>"]
notifier["Notifier<br/><i>plyer + email</i>"]
pipeline["Pipeline agent<br/><i>LangGraph</i>"]
orchestra["Orchestra agent<br/><i>LangGraph</i>"]
end

```

```

jenny --> betsyUI
finance --> devUI
betsyUI <--> api
devUI <--> api
api <--> db
scheduler --> pipeline
pipeline --> api
orchestra --> api
pipeline --> ollama
orchestra --> ollama
api --> notifier
notifier --> jenny
pipeline -.->|orders| suppliers

```

```

classDef person fill:#08427b,color:#fff,stroke:#073b6f;
classDef ext fill:#8a8a8a,color:#fff,stroke:#6b6b6b;
classDef container fill:#438dd5,color:#fff,stroke:#3a7cbf;
class jenny,finance person;
class suppliers,ollama ext;
class betsyUI,devUI,api,db,scheduler,notifier,pipeline,orchestra container;

```

Plain-English design docs (with diagrams) live in docs/ and pdf_exports/design/: gap-analysis (the before->after case for Betsy), pipeline-architecture, orchestra-architecture, and api-control-layer. All diagrams are in diagrams/ (see diagrams/00-INDEX.txt).

Setup

Prerequisites: Python 3.11+, Ollama running locally with llama3.1:8b pulled.

```

# install dependencies
pip install -r requirements.txt

# pull the model (if not already done)
ollama pull llama3.1:8b

# start the server
python run_server.py

```

Open <http://localhost:8000/betsy> for the agent UI (Jenny's view) or <http://localhost:8000> for the full dev dashboard.

Optional flags

```

python run_server.py --port 8080           # different port
python run_server.py --interval 2         # auto-run every 2 minutes
python run_server.py --no-reload          # disable hot reload

```

Project structure

betsy-ai-agent/	
??? server/	FastAPI app, routers, SQLite persistence
? ??? main.py	App entry point, lifespan, scheduler setup
? ??? db.py	SQLite write-through layer
? ??? state.py	In-memory state, scenario injection
? ??? scheduler_instance.py	APScheduler singleton
? ??? routers/	inventory, suppliers, orders, approvals, stats...
??? pipeline/	LangGraph pipeline (sequential)
? ??? graph.py	6-node workflow definition
? ??? nodes/	ingest -> detect -> evaluate -> decide -> act -> audit
??? orchestra/	LangGraph orchestra (parallel multi-agent)
??? dashboard/	
? ??? betsy.html	Agent UI -- for the operations manager
? ??? index.html	Dev dashboard -- full data view
??? mock_data/	12 SKUs, 6 suppliers, invoices, purchase orders
??? scenarios/	4 injectable test scenarios
??? tests/	
? ??? test_ema_learning.py	EMA formula verification
? ??? test_long_term_learning.py	Integration test -- 2 LLM runs, 8 delivery rounds
??? docs/	Architecture docs, API spec, test reports
??? decision_logs/	8 decision logs documenting every major build choice
??? run_server.py	Startup script

The two UIs

betsy.html -- designed for a non-technical operations manager. Plain English narrative, confidence shown as dots not decimals, approval cards that explain why Betsy stopped before asking for a decision. No jargon.

index.html -- full dev view. Raw inventory table, supplier scoreboard with live EMA scores, agent log, purchase order history.

Both surfaces read from the same API and update every 5 seconds.

Learning

Betsy gets smarter from delivery history. Every time a PO is marked delivered, the supplier's reliability score updates via an exponential moving average:

```
new_score = 0.2 × delivery_performance + 0.8 × old_score
performance = max(0.0, 1.0 - lateness_days × 0.1)
```

A supplier who delivers 8 days late repeatedly drops from a 0.92 score to 0.44 over 5 deliveries. That changed score changes who Betsy orders from next time -- demonstrated in tests/test_long_term_learning.py.

Decision logs

Eight decision logs in decision_logs/ document every major choice made during the build -- framework selection, architecture tradeoffs, UI design rationale, the HITL approval flow, persistence strategy, the learning mechanism, and notifications. Each log follows the DOT framework research format used in the ICT bachelor programme.

Log | What it covers

DL-01 | Building the mock environment

DL-02 | Pipeline vs Orchestra architecture

DL-03 | UI design for non-technical users

DL-04 | First real run and model selection

DL-05 | HITL approval queue end-to-end

DL-06 | SQLite persistence, EMA learning, APScheduler

DL-07 | Proving that score learning changes decisions

DL-08 | Desktop + email notifications

Running tests

```
# start the server first  
python run_server.py
```

```
# EMA formula verification (fast)  
python tests/test_ema_learning.py
```

```
# Long-term learning integration test (3-5 min, requires LLM)  
python tests/test_long_term_learning.py
```